Reto de recuperación — Corte 2

Estudiante: Santiago Carlosama Ortiz

Computación en Internet 3 · NestJS + TypeORM + PostgreSQL + JWT

Reglas

- Trabajas sobre **tu mismo repositorio del parcial**, en una rama nueva llamada `recuperacion`.
- Haz **un commit por cada punto** que completes, con un mensaje que diga qué hiciste. Se revisa el historial.
- La entrega se sustenta en persona (10 minutos): vas a explicar tu código y a hacer un cambio pequeño en vivo. Lo que no puedas explicar no cuenta completo.
- Usa imports relativos (`../student/entities/student.entity`), no `src/...`.

Parte A — Corrige tu parcial (40 pts)

Tu proyecto no compilaba. Esta parte es para que el módulo `program` funcione.

#	Qué corregir	Pts
A1	El proyecto compila: agrega los imports que faltan (`OneToMany`, `ApiProperty`, `IsString`, `IsArray`, ...) y usa `CreateProgramDto` en el controlador (la clase `CreateProgram` no existe).	4
A2	Agrega la columna `credits` (entero) a `Program` y valídala en el DTO con `@IsInt` y `@IsPositive`. `name` usa `@MinLength(3)`.	5
A3	Nombre o código repetido → `409`. Hoy `handleException` convierte el `23505` en un `500`.	6
A4	Agrega `GET /api/program/:id` (público) con `ParseUUIDPipe`, los estudiantes del programa y `404` si no existe.	6
A5	`PATCH /api/program/:id/students` con body `{ "studentIds": [...] }` (`@IsUUID('4', { each: true })`). Debe ser **acumulativo**: hoy `program.students = ...` reemplaza la lista y los anteriores quedan sin programa. Todas las operaciones con `queryRunner.manager`.	12
A6	Escrituras solo para `admin` (hoy está `@Auth(ValidRoles.teacher)`), y `createdBy` (`@ManyToOne(() => User)`) se llena con `@GetUser()`.	7

Parte B — Módulo `company` (40 pts)

Las empresas donde los estudiantes hacen práctica se cargan en lote desde un archivo que entrega la oficina de prácticas.

Entidad `Company`: `id` (UUID generado), `name` (texto, único), `nit` (texto, único, formato 9 dígitos, guion y 1 dígito, por ejemplo `890123456-7`), `sector` (texto), `createdBy` (`@ManyToOne(() => User)`).

#	Requisito	Pts
B1	Entidad y módulo registrados; la tabla se crea al arrancar.	5
B2	`CreateCompanyDto`: `name` con `@MinLength(3)`, `nit` con `@Matches(/^\d{9}-\d$/)`, `sector` con `@IsString`.	5
B3	`POST /api/company/bulk` (solo `admin`), body `{ "companies": [...] }`. Validación anidada (`@ValidateNested` + `@Type`): un elemento inválido o un lote vacío → `400`.	10
B4	El lote es **todo o nada**: si un NIT o nombre ya existe o se repite dentro del lote → `409` y no se crea ninguna. Responde `201` con las empresas creadas.	12
B5	Cada empresa guarda `createdBy` con `@GetUser()`. `GET /api/company` público y paginado; `GET /api/company/:id` con `ParseUUIDPipe` y `404`.	8

Sustentación (20 pts)

En persona, con tu código abierto. Se te pedirá explicar partes de tu solución y hacer un cambio pequeño en vivo.

Entrega

URL del repositorio y nombre de la rama (`recuperacion`).